

TECHNICAL DOCUMENTATION

Campus Event Recommendation System

Project handover and deployment documentation

Field	Value
Project team	Petar Acevski
Project repository	github.com/PetarAcevski/campus_event_recommendation_system
Document version	1.0
Date	September 2026
Project type	Campus Event Recommendation System

The project uses an explainable, deterministic scoring formula. No authentication and no machine learning model are required.

Contents

1. Introduction and Main Objectives
2. Prerequisites and Technologies Used
3. Technical Report and Architecture
4. Installation and Startup Steps
5. Configuration and Docker Compose
6. AI Components
7. User Manual and Functionality
8. API Documentation
9. Database Documentation
10. Recommendation Logic Explanation
11. Testing and Validation
12. Technical Limitations and Future Improvements
13. Glossary and Appendices

1. Introduction and Main Objectives

The Campus Event Recommendation System centralizes university events and recommends the most relevant ones to a student. The user can select a student profile, review ranked event cards, browse all events, and filter events by category, date, and free entry.

1.1 Main Functional Requirements

Area	Implemented functionality
Data processing	Imports 100 provided campus events and generates/imports 80 student profiles.
REST API	Students, events, recommendations, filters, and POST creation endpoints.
Recommendations	Interest, availability, audience, Haversine distance, affordability, and capacity scoring.
Frontend	Mobile-first Tailwind interface with selector, cards, states, filters, and Load more.

3. Technical Report and System Architecture

Frontend	FastAPI Backend	PostgreSQL
HTML + JavaScript + Tailwind CSS Student selector, cards, filters and responsive mobile UI	Routers, schemas, SQLAlchemy models, Haversine service and recommendation service	students and events tables with audit timestamps

Data flow: CSV files -> validation/import script -> PostgreSQL -> FastAPI endpoints -> frontend event cards.
--

2. Prerequisites and Technologies Used

Prerequisites for Project Implementation

Requirement	Purpose
Python and uv	Run the backend, scripts, dependency installation, and automated tests.
Docker Desktop	Run PostgreSQL, FastAPI, and Nginx frontend services using one command.
PostgreSQL	Persistent relational database used by the application.
Git and GitHub	Version control and project repository management.
Modern browser	Use the mobile frontend and Swagger API documentation.

Technologies Used

Area	Technology	Usage
Backend	Python, FastAPI, Uvicorn	REST API, validation, Swagger/OpenAPI documentation.
Database	PostgreSQL, SQLAlchemy, psycopg2-binary	Persistent storage and ORM database access.
Frontend	HTML, JavaScript, Tailwind CSS	Mobile-first frontend, cards, filters, and UI states.
Utilities	uv, python-dotenv, pytest, httpx	Environment management, configuration, and automated tests.
Deployment	Docker Compose, Nginx	Single-command local deployment and reverse proxy.

2.1 Data Preparation

The supplied campus_events_skopje CSV is stored as data/raw/campus_events.csv. It includes 100 events. A deterministic Python script generates 80 synthetic student profiles with multiple comma-separated interests.

9. Database Documentation

Table	Purpose	Key fields
students	Stores student profiles and recommendation preferences.	id, name, email, faculty, year_of_study, latitude, longitude, interests, available_from, available_to, max_price
events	Stores campus events used for filtering and ranking.	id, title, category, organizer, location, coordinates, start/end, capacity, registered_count, audience, price, is_online

Audit Fields and Data Integrity

Both database tables contain created_at, updated_at, and deleted_at fields. The import script validates required values, date/time formats, boolean values, capacity, and registered_count. Records are inserted with SQLAlchemy session.merge(), allowing safe repeat imports based on the record ID.

Dataset Summary

Dataset	Count	Source
Campus events	100	Provided campus_events_skopje CSV file
Student profiles	80	Synthetic profiles generated with Python

4. Installation and Startup Steps

All commands are executed from the repository root: campus-event-recommendation-system/.

Step	Command / Action
1. Clone repository	<code>git clone https://github.com/PetarAcevski/campus_event_recommendation_system.git</code>
2. Install dependencies	<code>uv sync</code>
3. Configure environment	Create <code>.env</code> with <code>DATABASE_URL</code> for the local PostgreSQL database.
4. Local database setup	<code>uv run python -m scripts.create_tables</code>
5. Import data	<code>uv run python -m scripts.import_data</code>
6. Run with Docker	<code>docker compose up --build</code>

5. Configuration and Docker Compose

The `DATABASE_URL` environment variable defines the SQLAlchemy connection. Docker Compose configures the database service with the `campus_app` user and `campus_events` database. The frontend container uses Nginx and forwards `/api` requests to the backend container.

Setting	Example
<code>DATABASE_URL</code>	<code>postgresql+psycopg2://campus_app:campus_dev_password@db:5432/campus_events</code>
Frontend URL	<code>http://127.0.0.1:8080</code>
Swagger URL	<code>http://127.0.0.1:8080/api/docs</code>
Docker services	<code>db, backend, frontend</code>

Docker Compose creates the tables and imports the CSV data before starting Uvicorn. The PostgreSQL volume persists data between container restarts.

7. User Manual and Functionality

The user opens the mobile frontend, selects a student profile, and receives ranked event cards. Each card shows title, category, organizer, location, date/time, distance, price, available places, score, and an explanation. Only five cards are shown initially; the Load more button displays the next five recommendations.

8. FastAPI REST API

FastAPI exposes the system functionality as REST endpoints. Swagger documentation is available through Docker at `/api/docs` and is used for manual endpoint testing.

Method	Endpoint	Description
GET	<code>/health</code>	Checks API availability and PostgreSQL connection.
GET	<code>/students</code>	Returns all active student profiles.
GET	<code>/students/{student_id}</code>	Returns a selected student.
POST	<code>/students</code>	Creates a new student profile.
GET	<code>/events</code>	Returns events with optional filters.
GET	<code>/events/{event_id}</code>	Returns a selected event.
POST	<code>/events</code>	Creates a new event.
GET	<code>/recommendations/{student_id}</code>	Returns ranked recommendations for a stored student.
GET	<code>/recommendations?lat=...&lon;=...&interests;=...</code>	Returns recommendations for a custom location and interests.

Event Filters

The GET `/events` endpoint supports combinable query parameters: category, date, and free_only. Examples: `/events?category=Technology`, `/events?date=2026-09-23`, and `/events?category=Technology&free;_only=true`.

The frontend filter form sends the selected values directly to GET `/events` and displays the returned event cards.

6. AI Components

No runtime AI model, machine-learning model, external AI API, model training, vector database, or AI key is used in this project. The recommendation system is intentionally deterministic and rule-based, as required by the project brief.

Component	Status	Explanation
Machine-learning model	Not used	There is no model training or inference step.
External AI API	Not used	No OpenAI key or other external AI service is configured.
Recommendation logic	Implemented	Transparent weighted rules calculate every event score.

10. Recommendation Logic Explanation

The algorithm is deterministic: the same student and event data always produce the same result. Full events and events that exceed a student's max_price are excluded before ranking. Eligible events are scored and sorted by score descending, distance ascending, and start date ascending.

Scoring factor	Maximum	Rule
Interest match	30	Event category matches one of the student's interests.
Time availability	20	Full time match: 20; event starts during availability: 10.
Faculty / audience	15	Event targets the student's faculty or All Students.
Distance	15	Calculated using Haversine; online events receive 15.
Price	10	Free: 10; affordable paid event: 5.
Capacity	10	Higher ratio of available places earns more points.
Total	100	Final explainable recommendation score.

Haversine Distance

The Haversine formula calculates physical distance in kilometres using the latitude and longitude of the student and event. Distance scores are 15 points up to 0.5 km, 12 up to 1 km, 9 up to 3 km, 6 up to 5 km, and 2 above 5 km.

Every recommendation response includes a short reason, for example: 'Matches your Technology interest. Fits your available time. Very close to your location. Free event.'

5. Frontend, Docker and Testing

Frontend Implementation

The mobile-first frontend is built in one index.html file with Tailwind CSS and JavaScript. It includes a Campus logo, Montserrat font, student selector, responsive recommendation cards, loading/error/empty states, event filters, and a Load more button that displays recommendations in groups of five.

Docker Compose

Service	Responsibility
db	PostgreSQL 16 database with a persistent named volume.
backend	FastAPI application, table creation, and idempotent data import.
frontend	Nginx server for the mobile UI and /api reverse proxy to FastAPI.

Testing Evidence

Test type	Evidence
Automated tests	uv run python -m pytest completed successfully with 8 passed tests.
API tests	Swagger used to test health, students, events filters, and recommendations endpoints.
Frontend tests	Student selection, filtered events, mobile cards, Load more, and API data loading verified in browser.
Database tests	PostgreSQL counts confirmed: 80 students and 100 events.

Repository and Local Run

The project repository is maintained on GitHub. Local Docker run command: docker compose up --build. The frontend is available at <http://127.0.0.1:8080> and Swagger at <http://127.0.0.1:8080/api/docs>.

Conclusion: the project satisfies the required stack and implements a complete locally running Campus Event Recommendation System with explainable event recommendations.

12. Technical Limitations and Future Improvements

Current limitation	Possible future improvement
Synthetic student profiles	Import real student preferences only with consent and appropriate privacy controls.
No event registration workflow	Add event registrations, attendance status, and available-seat updates.
Rule-based weights	Allow controlled user feedback to tune the deterministic scoring weights.
No authentication	Add secure user authentication and profile ownership in a future version.
Local deployment	Deploy the Docker Compose stack to a managed server or cloud environment.

13. Glossary

Term	Meaning
API	Application Programming Interface used by the frontend to request backend data.
Docker Compose	Tool that starts the PostgreSQL, backend, and frontend containers together.
Haversine formula	Formula used to calculate distance between latitude/longitude coordinates.
ORM	SQLAlchemy maps Python models to PostgreSQL database tables.
Recommendation score	Deterministic 0-100 value used to rank eligible events.
Swagger UI	Interactive FastAPI API documentation used for manual testing.

Appendix: Project File Structure

backend/ contains FastAPI routers, models, schemas, and services. data/raw/ contains campus_events.csv and students.csv. scripts/ contains table creation, data generation, and import scripts. frontend/ contains the mobile interface, logo assets, Nginx configuration, and Dockerfile. tests/ contains automated pytest coverage.